ANKARA ÜNİVERSİTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BLM4522 - Ağ Tabanlı Paralel Dağıtım Sistemleri

Metehan ULUOCAK - 21290433

Github linki : https://github.com/Metehan-Uluocak/BLM4522-Ag-Tabanli-Paralel-Dagitim-Sistemleri

İÇİNDEKİLER

1. Veritabanı Performans Optimizasyonu ve İzleme

1.1. Amaç 3

1.2. Yapılacaklar 3

1.2.1. Veritabanı İzleme Modülü 3

1.2.2. Sorgu Analizi ve İyileştirme 3

1.2.3. İndeks Yönetimi 3

1.2.4. Performans Raporlama ve Görselleştirme 4

1.2.5. Roller ve Yetkilendirme 4

1.2.5.1. Admin 4

1.2.5.2. Analist 4

1.2.5.3. Uygulama Kullanıcısı 4

1.3. Kullanılacak Araçlar 4

1.4. Proje Planı 5

2. Veritabanı Yedekleme ve Felaketten Kurtarma 6

2.1. Amaç 6

2.2. Yapılacaklar 6

2.2.1. Yedekleme Stratejisi Modülü 6

2.2.2. Tam Yedekleme ve Arşivleme 6

2.2.3. Otomasyon ve Zamanlama 6

2.2.4. Felaket Senaryosu ve Geri Dönüş Testi 6

2.2.5. Sonuç Raporlama ve Doğrulama 7

2.2.6. Roller ve Yetkilendirme 7

2.2.6.1. Admin 7

2.2.6.2. Operasyon Sorumlusu 7

2.2.6.3. Denetçi 7

2.3. Kullanılacak Araçlar 7

2.4. Proje Planı 8

Veritabanı Performans Optimizasyonu ve İzleme

AMAÇ

Bu projenin amacı, büyük veri hacmine sahip bir veritabanında yavaş çalışan sorguları tespit ederek performans darboğazlarını gidermek, veritabanı kaynak kullanımını optimize etmek ve sistemin sürdürülebilir şekilde izlenmesini sağlamaktır.

Proje kapsamında sorgu performansı düzenli olarak ölçülecek, uygun indeksleme stratejileri uygulanacak ve iyileştirme öncesi/sonrası sonuçlar karşılaştırmalı olarak raporlanacaktır.

Yapılacaklar

Aşağıdaki maddeler proje kapsamında geliştirilecek temel işlevlerdir.

1. **Veritabanı İzleme Modülü**

    - Sistem üzerinde çalışan sorguların süre, maliyet ve kaynak tüketimleri izlenir.
    - `pg_stat_statements` ve `EXPLAIN (ANALYZE, BUFFERS)` çıktıları kullanılarak yavaş sorgular tespit edilir.
    - En fazla kaynak tüketen sorgular periyodik olarak raporlanır.
    - Sorgu bazlı CPU/IO etkileri değerlendirilir

2. **Sorgu Analizi ve İyileştirme**

    - En az 3 adet kritik yavaş sorgu seçilir.
    - Her sorgu için mevcut yürütme planı analiz edilir.
    - `SELECT *` yerine gerekli sütunlar kullanılarak sorgu sadeleştirilir.
    - Filtre ve JOIN koşulları optimize edilerek sorgu maliyeti düşürülür.
    - İyileştirme öncesi/sonrası süre karşılaştırmaları tablo halinde sunulur.

3. **İndeks Yönetimi**

    - WHERE ve JOIN alanları için uygun indeks stratejisi belirlenir.
    - Gerekli durumlarda birleşik (composite) indeksler uygulanır.
    - Kullanılmayan veya verimsiz indeksler tespit edilerek kaldırılır.
    - İndeksleme sonrası yazma/okuma performans dengesi değerlendirilir.

4. Performans Raporlama ve Görselleştirme

- Kritik sorgular için önce/sonra performans metrikleri toplanır.
- Sorgu süreleri, maliyetler ve okunan blok sayıları raporlanır.
- Sonuçlar tablo veya grafik formatında görselleştirilir.
- Proje sonunda performans kazanımı yüzdesi açık şekilde belirtilir.

5.Roller ve Yetkilendirme

Sistem üç farklı kullanıcı rolü içerecektir.

Admin

- Veritabanı performans raporlarının tamamına erişebilir.
- İndeks oluşturma/silme işlemlerini yönetebilir.
- İzleme araçlarının yapılandırmasını değiştirebilir.

Analist

- Sorgu performans raporlarını görüntüleyebilir.
- İyileştirme önerileri hazırlayabilir.
- DDL işlemleri (tablo/indeks değişikliği) yapamaz.

Uygulama Kullanıcısı

- Sadece uygulama sorgularını çalıştırabilir.
- Performans raporlarına ve yönetim işlemlerine erişemez.

Kullanılacak Araçlar

- PostgreSQL
- pgAdmin 4
- `pg_stat_statements`
- `EXPLAIN (ANALYZE, BUFFERS)`

Proje Planı

Adım 1

- 1 milyon+ satır içeren örnek bir veri seti seç.
- Verileri PostgreSQL içine aktar
- Veri tiplerini ve eksik değerleri kontrol et.

Adım 2

- `pg_stat_statements` eklentisini etkinleştir.
- Kritik sorguları belirle ve sürelerini kaydet.
- İlk ölçüm tablosunu oluştur.

Adım 3

- Her yavaş sorgu için `EXPLAIN (ANALYZE, BUFFERS)` çalıştır.
- `Seq Scan`, yüksek maliyetli `Sort`, kötü `Join` planlarını not et.

Adım 4

- WHERE ve JOIN sütunlarına uygun indeksleri oluştur.
- Gerekirse birleşik (composite) indeks dene.
- Gereksiz veya kullanılmayan indeksleri kaldır.

Adım 5

- `SELECT *` yerine gerekli sütunları seç.
- Filtreleri erken uygula.
- Maliyetli alt sorguları daha uygun yapıya dönüştür.

Adım 6

- `role_admin`, `role_analyst`, `role_app_user` rollerini oluştur.
- En az yetki ilkesiyle izinleri tanımla.

Adım 7

- Aynı sorguları tekrar çalıştır.
- Önce/sonra süre ve maliyet farkını tablo ve grafikle göster

Veritabanı Yedekleme ve Felaketten Kurtarma

AMAÇ

Bu projenin amacı, veritabanında oluşabilecek veri kaybı ve sistem kesintisi risklerine karşı güvenilir bir yedekleme mimarisi kurmak, geri dönüş süreçlerini test ederek doğrulamak ve sürekliliği garanti altına almaktır.

Proje kapsamında tam yedekleme, WAL arşivleme ve belirli bir zamana geri dönüş (Point-in-Time Restore) adımları uygulanacak; test sonuçları RTO ve RPO metrikleri ile birlikte raporlanacaktır.

Yapılacaklar

Aşağıdaki maddeler proje kapsamında geliştirilecek temel işlevlerdir.

1. **Yedekleme Stratejisi Modülü**
   - Sistem için yedekleme politikası tanımlanır.
   - RTO (geri dönüş süresi) ve RPO (kabul edilebilir veri kaybı süresi) hedefleri belirlenir.
   - Tam yedekleme sıklığı ve saklama süreleri planlanır.
   - Kritik tablolar ve öncelikli veri alanları sınıflandırılır.
2. **Tam Yedekleme ve Arşivleme**
   - `pg_dump` ile mantıksal tam yedekleme uygulanır.
   - Gerekli senaryolarda `pg_basebackup` ile fiziksel yedekleme planlanır.
   - WAL arşivleme yapılandırması etkinleştirilir.
   - Yedek dosyalarının bütünlüğü ve erişilebilirliği doğrulanır.
3. **Otomasyon ve Zamanlama**
   - Yedekleme işlemleri Task Scheduler ile otomatik hale getirilir.
   - Günlük/haftalık yedek görevleri ayrı planlanır.
   - Başarısız yedekleme durumları için log ve uyarı mekanizması eklenir.
   - Yedekleme geçmişi düzenli olarak raporlanır.

4. Felaket Senaryosu ve Geri Dönüş Testi

- Kontrollü veri silme veya bozulma senaryosu oluşturulur.
- Olay zamanı kayıt altına alınır.
- Belirlenen zamana geri dönüş (PITR) adımları uygulanır.
- Geri yükleme sonrası veri tutarlılığı test edilir.

5. Sonuç Raporlama ve Doğrulama

- Yedekleme başarı oranı, geri dönüş süresi ve veri kaybı miktarı raporlanır.
- Ön test ve son test karşılaştırmaları tablo halinde sunulur.
- Elde edilen sonuçlara göre iyileştirme önerileri hazırlanır.
- Nihai çıktılar ekran görüntüleri ve log kayıtları ile desteklenir.

6. Roller ve Yetkilendirme

Sistem üç farklı kullanıcı rolü içerecektir.

Admin

- Yedekleme politikalarını oluşturabilir ve değiştirebilir.
- Geri yükleme işlemlerini başlatabilir.
- Tüm log ve sistem raporlarına erişebilir.

Operasyon Sorumlusu

- Planlı yedekleme görevlerini izleyebilir.
- Başarısız görevler için ilk müdahaleyi yapabilir.
- Politika değişikliği yapamaz.

Denetçi

- Yedekleme ve geri dönüş raporlarını görüntüleyebilir.
- Uyumluluk ve doğrulama kontrollerini yapabilir.
- Yedek alma veya geri yükleme işlemi başlatamaz.

Kullanılacak Araçlar

- PostgreSQL
- pgAdmin 4
- `pg_dump`, `pg_restore`, `pg_basebackup`
- Windows Task Scheduler

Proje Planı

Adım 1

- RPO hedefini belirle.
- RTO hedefini belirle.
- Yedek saklama süresini tanımla.

Adım 2

- `pg_dump` ile mantıksal tam yedek al.
- Kritik ortam için `pg_basebackup` planı oluştur.

Adım 3

- `archive_mode = on` yapılandır.
- `archive_command` tanımla.
- WAL dosyalarının güvenli klasöre yazıldığını doğrula.

Adım 4

- Task Scheduler ile günlük yedek görevi oluştur.
- Başarısız yedeklemeler için log/uyarı mekanizması ekle.

Adım 5

- Test tablosunda kontrollü veri silme işlemi yap.
- Silme anını zaman damgası ile kaydet.
- Hedef zamana geri dönüş (PITR) uygula.

Adım 6

- Satır sayısı ve kritik sorguları karşılaştır.
- Uygulama tarafında veri tutarlılığını kontrol et.

Adım 7

Yedek listesi, boyutlar, geri dönüş süresi tablosu hazırla.